
UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS
FACULTAD DE INGENIERÍA



PROYECTO INTEGRADOR FINAL

Sistema Punto de Venta MVC

(Java Swing & SQL Server)

**DOCUMENTO COMPLETO DE ANÁLISIS DE REQUERIMIENTOS,
DISEÑO E INGENIERÍA DE SOFTWARE**

Programa Educativo: Ingeniería en Sistemas Computacionales
Asignatura: Programación Avanzada / Ingeniería de Software
Patrón de Diseño: Modelo-Vista-Controlador (MVC)
Alumnos: Rafael Ricardez Reyes
Jorge Airam Cervantes Morán
Grupo: Grupo M
Base de Datos: Microsoft SQL Server Express Edition

Tampico, Tamaulipas, México
17 de mayo de 2026

Índice

1	Introducción	3
1.1	Propósito del Documento	3
1.2	Alcance del Producto	3
1.3	Definiciones, Siglas y Abreviaturas	3
2	Descripción General	4
2.1	Descripción del Proceso de Punto de Venta (Evidencia de Campo)	4
2.2	Usuarios Finales y Características	4
2.3	Entorno de Operación	4
3	Requerimientos Funcionales	5
3.1	Flujo Básico del Sistema	5
3.2	Casos de Uso	6
3.2.1	Caso de Uso: Registrar Venta con Descuento	6
3.2.2	Caso de Uso: Registrar Devolución de Producto	7
4	Diccionario de Datos de las Clases y Base de Datos	8
4.1	Diccionario de la Clase Modelo "Producto" y Tabla "productos"	8
4.2	Diccionario de la Clase Modelo "Venta" y Tabla "ventas"	8
4.3	Diccionario de la Clase Modelo "DetalleVenta" y Tabla "detalle_ventas"	8
5	Modelo UML de Clases	9
5.1	Diagrama de Clases (Modelado Vectorial Nativo)	9
6	Modelo UML de Actividades	10
6.1	Diagrama de Actividades con Swimlanes: Flujo de Cobro	10
7	Modelo UML de Secuencias	11
7.1	Diagrama de Secuencia: Finalizar Venta	11
8	Relaciones Detectadas entre Clases	12
8.1	Herencia (Generalización)	12
8.2	Asociación	12
8.3	Composición	12
8.4	Agregación	12
9	Interfaces de Usuario (Descripción de Funcionalidad)	13
9.1	LoginVista	13
9.2	MainVista (Dashboard)	13
9.3	VentasVista	13
9.4	AlmacenVista	13
10	Flujo de las Interfaces de Usuario por cada Requerimiento Funcional	14
10.1	Flujo de Alta y Edición de Productos (RF-1)	14
10.2	Flujo de Control de Almacén y Ajuste de Stock (RF-2)	14
10.3	Flujo de Venta y Despliegue de Ticket (RF-3)	15

11 Requerimientos No Funcionales	16
11.1 Seguridad	16
11.2 Usabilidad	16
11.3 Hardware y Eficiencia	16
12 Manual de Usuario (Instalación y Operación)	17
12.1 Guía de Instalación del Sistema	17
12.1.1 Configuración de la Base de Datos	17
12.1.2 Configuración del Proyecto en Java	17
12.1.3 Ejecución	17
12.2 Guía de Uso Diario del Sistema	17
13 Documento Completo de Análisis de Requerimientos	19
13.1 Matriz de Trazabilidad de Requerimientos mínimos	19
13.2 Sistema de Auditoría y Manejo de Errores (Log)	19
14 Explicación del Uso de Inteligencia Artificial en el Proyecto	20
14.1 Justificación del Uso de la IA	20
14.2 Lista de Prompts Utilizados en el Desarrollo	20
15 Conclusiones y Recomendaciones	22
15.1 Conclusiones	22
15.2 Recomendaciones	22
15.3 Bibliografía	22

1 Introducción

1.1 Propósito del Documento

Este documento de análisis, diseño y especificación técnica tiene como propósito formalizar de manera rigurosa y académica el ciclo de vida del desarrollo del “Sistema Punto de Venta MVC” (PuntoVentaMVC_Z). A través de este escrito, se detallan las metodologías aplicadas, las decisiones arquitectónicas bajo el patrón de diseño MVC, el modelo entidad-relación de la base de datos de Microsoft SQL Server, y el uso justificado de herramientas de inteligencia artificial como catalizador de productividad.

1.2 Alcance del Producto

El sistema es una aplicación de escritorio multiplataforma robusta diseñada para sistematizar la operación diaria de tiendas de abarrotes de tamaño mediano que manejan un inventario de más de 150 productos. El alcance técnico incluye:

- Registro y mantenimiento de productos con almacenamiento dinámico de rutas de imágenes locales.
- Gestión automatizada del almacén mediante entradas y salidas síncronas de stock físico.
- Módulo de ventas con validación automatizada de stock disponible, agregación de ítems, cálculo automático de impuestos y descuentos, y emisión de comprobantes impresos (Tickets).
- Módulo de devoluciones integrado que reincorpora automáticamente los productos devueltos al almacén general.
- Analítica avanzada a través de exportaciones de datos en formatos nativos de Microsoft Excel (mediante la API Apache POI) y PDF (mediante OpenPDF).

1.3 Definiciones, Siglas y Abreviaturas

UML: Unified Modeling Language (Lenguaje Unificado de Modelado).

MVC: Modelo-Vista-Controlador.

JDBC: Java Database Connectivity.

POI: Apache Poor Obfuscation Implementation.

DAO: Data Access Object (Objeto de Acceso a Datos).

POO: Programación Orientada a Objetos.

2 Descripción General

2.1 Descripción del Proceso de Punto de Venta (Evidencia de Campo)

Para fundamentar las necesidades del software, se llevó a cabo una visita de campo y levantamiento de requisitos en el negocio local "Abarrotes El Competidor". A continuación, se expone cómo trabaja el negocio actualmente en relación con los requisitos mínimos solicitados:

1. **Catálogo de Productos y Registro de Mercancía:** El negocio maneja un catálogo de más de 200 productos. Actualmente, la entrada de mercancía nueva se registra manualmente en una libreta cuando llega el proveedor. Al no tener un sistema ágil, no se puede verificar si el costo del producto aumentó de inmediato.
2. **Gestión de Precios:** Cuando los precios cambian debido a la inflación, el dueño debe revisar los estantes uno por uno y colocar etiquetas de precios hechas a mano, lo que resulta lento e impreciso.
3. **Módulo de Compras/Ventas:** El cobro se realiza con una calculadora básica de mano. El cajero debe memorizar los precios de las verduras y de los productos a granel. Al finalizar, entrega un recibo de papel genérico sin desglose de impuestos ni detalles del producto.
4. **Control de Stock y Límites:** El inventario se revisa de manera visual al final del día. Si un producto de alta demanda se agota antes, el negocio pierde la venta por falta de alerta oportuna.
5. **Devoluciones:** Las devoluciones se manejan entregando efectivo directamente de la caja registradora, pero no se realiza ningún registro administrativo del motivo, ni se contabiliza si el producto regresó dañado o si puede volver al inventario para su venta.

2.2 Usuarios Finales y Características

- **Administrador:** Encargado de la supervisión global, alta de productos, definición de stock mínimo, actualización de precios de venta y análisis financiero a través de la exportación de reportes mensuales.
- **Vendedor / Cajero:** Enfocado en la atención directa y ágil al cliente. Requiere una interfaz rápida para agregar productos, cobrar y generar el comprobante físico (ticket) sin demoras.

2.3 Entorno de Operación

El sistema opera de forma local sobre la máquina virtual de Java (JRE 11 o superior). Utiliza Microsoft SQL Server Express localmente para asegurar transacciones síncronas rápidas, tolerantes a fallos y sin necesidad de conexión activa a Internet.

3 Requerimientos Funcionales

3.1 Flujo Básico del Sistema

El sistema coordina de manera síncrona los siguientes subprocesos de negocio:

1. El usuario inicia sesión ingresando su usuario y clave en la interfaz LoginVista.
2. El sistema carga el Dashboard principal (MainVista) mostrando estadísticas generales (Productos, Ventas, Ingresos Totales y Alerta de Stock Bajo) actualizadas en tiempo real.
3. Al procesar una venta, el cajero digita el ID del producto y la cantidad. El sistema valida el inventario en milisegundos. Si hay suficiente stock, calcula los subtotales y muestra el producto en la tabla temporal.
4. Al dar clic en "Finalizar Venta", el sistema registra la venta en la base de datos, reduce el inventario, genera e imprime el ticket de compra y limpia la pantalla.

3.2 Casos de Uso

3.2.1 Caso de Uso: Registrar Venta con Descuento

Caso de Uso: Registrar Venta con Descuento

Actor:	Cajero
Precondiciones:	El usuario está autenticado y la caja de ventas está activa.
Flujo Principal:	1. El cajero ingresa el ID del producto y la cantidad. 2. El sistema valida la existencia física en inventario mediante <code>obtenerProductoPorId()</code>. 3. Se repite el proceso para todos los productos de la compra. 4. El cajero aplica un descuento porcentual en la casilla correspondiente. 5. El sistema calcula y muestra de forma dinámica los totales (Subtotal, Descuento y Neto a pagar). 6. El cajero presiona "Finalizar Venta". 7. El sistema registra la cabecera en la tabla <code>ventas</code> y sus detalles en <code>detalle_ventas</code>. 8. El sistema descuenta el stock físico de la tabla <code>productos</code>. 9. Se despliega en pantalla la interfaz del ticket de compra.
Flujo Alternativo:	En el paso 2, si la cantidad solicitada supera el stock disponible en la base de datos, el sistema bloquea la transacción y muestra una alerta mediante <code>ErrorControlador.mostrarAdvertencia()</code> .
Postcondiciones:	Transacción registrada, inventario descontado y ticket generado.

3.2.2 Caso de Uso: Registrar Devolución de Producto

Caso de Uso: Registrar Devolución de Producto

Actor:	Administrador / Cajero Autorizado
Precondiciones:	El ID de venta y el producto deben existir en las tablas históricas.
Flujo Principal:	1. El usuario accede al módulo de Devoluciones. 2. Selecciona la venta correspondiente de la tabla de historial. 3. Digita el ID del producto, la cantidad a devolver y el motivo. 4. El sistema calcula el monto a reembolsar y guarda la devolución en la tabla devoluciones. 5. El sistema incrementa el stock del producto mediante <code>agregarStock()</code> en la base de datos.
Postcondiciones:	El stock físico se actualiza y la devolución queda registrada para auditoría financiera.

4 Diccionario de Datos de las Clases y Base de Datos

Para asegurar la integridad de la información, se define el diccionario de datos de las clases modelo de Java y su correspondencia exacta con las tablas del motor relacional SQL Server.

4.1 Diccionario de la Clase Modelo “Producto” y Tabla “productos”

Cuadro 1: Mapeo de Datos de la Entidad Producto.

Atributo Java	Visibilidad	Campo SQL	Tipo SQL	Descripción / Restricción
id	private	id	INT IDENTITY	Clave Primaria. Autoincremental.
nombre	private	nombre	VARCHAR(100)	Nombre del producto. No Nulo.
precio	private	precio	DECIMAL(10,2)	Precio unitario de venta.
stock	private	stock	INT	Cantidad disponible en almacén.
categoria	private	categoria	VARCHAR(50)	Categoría del producto.
stockMinimo	private	stock_minimo	INT	Límite inferior de stock permitido.
imagenPath	private	imagen_path	VARCHAR(255)	Ruta local en disco de la imagen.

4.2 Diccionario de la Clase Modelo “Venta” y Tabla “ventas”

Cuadro 2: Mapeo de Datos de la Entidad Venta.

Atributo Java	Visibilidad	Campo SQL	Tipo SQL	Descripción / Restricción
id	private	id	INT IDENTITY	Clave Primaria. Folio de la venta.
fecha	private	fecha	DATE	Fecha de venta (asigna el servidor).
hora	private	hora	TIME	Hora exacta del registro de venta.
total	private	total	DECIMAL(10,2)	Monto total de la venta (Subtotal bruto).
descuento	private	descuento	DECIMAL(10,2)	Monto de descuento total aplicado.
montoPagar	private	monto_pagar	DECIMAL(10,2)	Neto final pagado por el cliente.

4.3 Diccionario de la Clase Modelo “DetalleVenta” y Tabla “detalle_ventas”

Cuadro 3: Mapeo de Datos de la Entidad DetalleVenta.

Atributo Java	Visibilidad	Campo SQL	Tipo SQL	Descripción / Restricción
idVenta	private	id_venta	INT	Clave Foránea vinculada a ventas (id).
idProducto	private	id_producto	INT	Clave Foránea vinculada a productos (id).
cantidad	private	cantidad	INT	Cantidad de unidades de este producto.
precio	private	precio	DECIMAL(10,2)	Precio unitario de este producto en la venta.
subtotal	private	subtotal	DECIMAL(10,2)	Subtotal calculado de este producto.

5 Modelo UML de Clases

Este diagrama muestra detalladamente las clases estructurales de la aplicación, organizadas bajo el patrón Modelo-Vista-Controlador (MVC), indicando sus variables miembro y operaciones clave.

5.1 Diagrama de Clases (Modelado Vectorial Nativo)

A continuación se presenta el modelado de clases generado dinámicamente con TikZ, mostrando de forma visual las herencias, composiciones y métodos esenciales en un formato estilizado naranja y negro. Se ha redimensionado de forma responsiva para evitar cualquier corte:

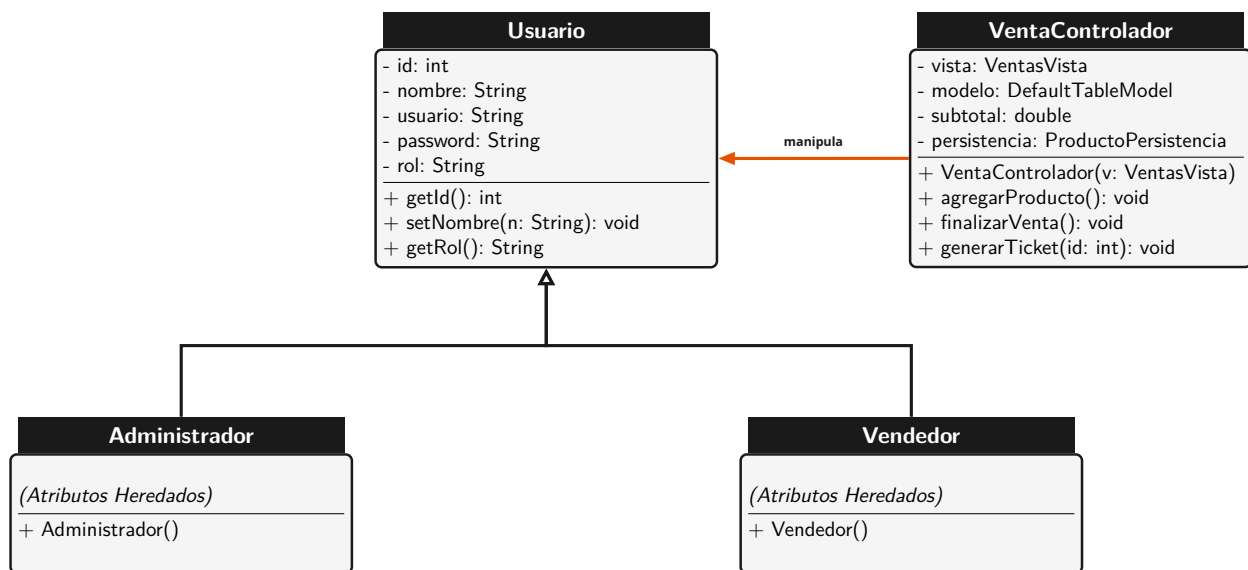


Figura 1: Diagrama de Clases UML del Sistema.

6 Modelo UML de Actividades

Este diagrama de actividades muestra los flujos lógicos y transaccionales del proceso de cobro y de validación de stock, organizados en carriles (Swimlanes) para separar las tareas del usuario de las del sistema y del servidor de base de datos.

6.1 Diagrama de Actividades con Swimlanes: Flujo de Cobro

El diagrama se ha adaptado proporcionalmente para encajar de manera óptima dentro de los márgenes geométricos de la hoja:

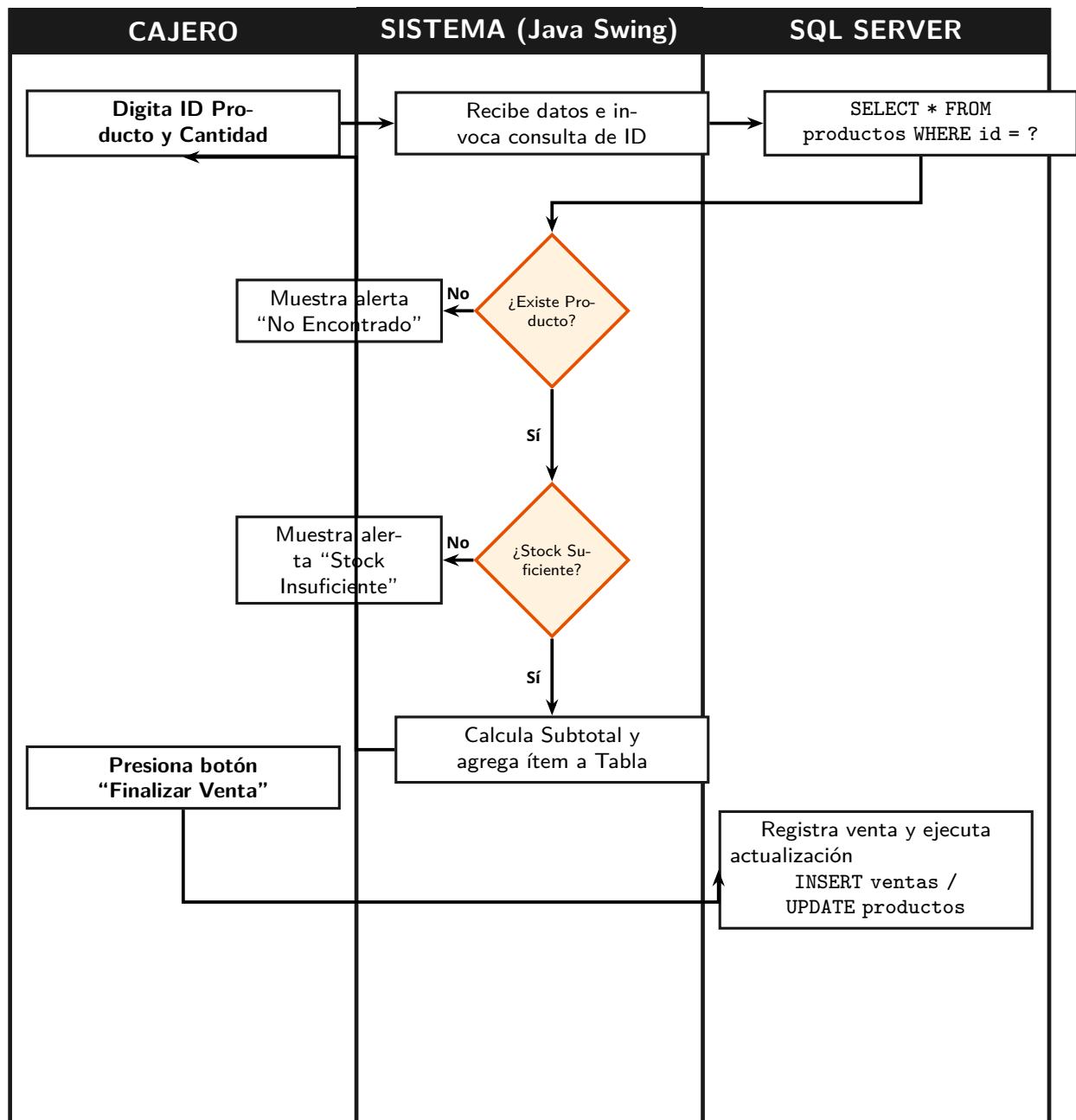


Figura 2: Diagrama de Actividades Transaccionales con Carriles de Responsabilidad.

7 Modelo UML de Secuencias

Este diagrama describe la secuencia temporal de las llamadas a métodos que ocurren entre los diferentes componentes de la aplicación durante el proceso de "Finalizar Venta".

7.1 Diagrama de Secuencia: Finalizar Venta

Se ha aplicado escalamiento vectorial adaptativo para evitar desbordes laterales:

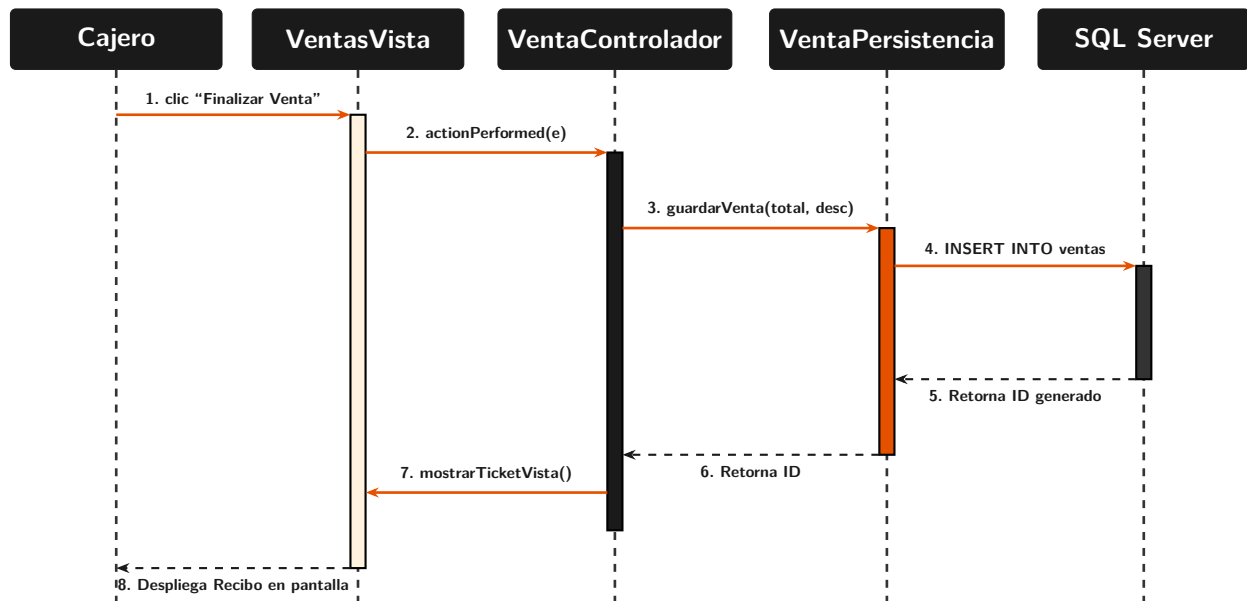


Figura 3: Diagrama de Secuencia de la Operación "Finalizar Venta".

8 Relaciones Detectadas entre Clases

Para justificar técnicamente la programación orientada a objetos del sistema de la tienda de abarrotes, se detallan las relaciones estructurales clave identificadas en el código fuente:

8.1 Herencia (Generalización)

La herencia se aplica en el subsistema de usuarios. Las clases `Administrador.java` y `Vendedor.java` heredan directamente de la clase base `Usuario.java` utilizando la palabra reservada `extends`:

Administrador Hereda de → Usuario

Esto permite reutilizar propiedades comunes (ID, Nombre, Usuario, Contraseña, Rol) y facilita la implementación de un control de acceso selectivo basándose en el polimorfismo de la propiedad `rol`.

8.2 Asociación

Existe una relación de dependencia síncrona entre los controladores y las vistas. Por ejemplo, el controlador `AlmacenControlador` tiene una asociación directa con `AlmacenVista` declarada como un atributo miembro privado:

Declaración de Asociación en `AlmacenControlador.java`

```
public class AlmacenControlador {  
    private AlmacenVista vista; // Asociacion directa síncrona  
}
```

Esto permite al controlador suscribirse a los eventos de los botones y manipular los campos de entrada de la interfaz gráfica de forma acoplada pero estructurada.

8.3 Composición

La relación entre una `Venta` y su `DetalleVenta` es de composición estricta. Un detalle de venta no tiene sentido de existencia lógica fuera del contexto de una transacción de cabecera que contenga un folio único de factura o ticket. Si la entidad `Venta` es eliminada, todos sus detalles asociados también se eliminan para mantener la integridad referencial de los datos en SQL Server.

8.4 Agregación

La persistencia y manipulación del inventario de `Producto` se relaciona con el carrito de compra de la venta mediante agregación. Los productos son independientes de la venta en sí: si se cancela o elimina una venta del historial, los productos que formaban parte de ella siguen existiendo en el catálogo de la tienda de abarrotes sin sufrir modificaciones estructurales, simplemente variando sus unidades disponibles de inventario físico.

9 Interfaces de Usuario (Descripción de Funcionalidad)

Las interfaces de usuario se diseñaron de manera limpia utilizando componentes Swing de Java, optimizando la pantalla para una operación rápida en la tienda de abarrotes.

9.1 LoginVista

Pantalla de acceso seguro de tamaño 900×540 píxeles. El panel izquierdo tiene un fondo gris oscuro institucional y muestra información de la versión del software. El panel derecho contiene campos específicos para ingresar el nombre de usuario y la contraseña de acceso.

9.2 MainVista (Dashboard)

Es el panel de control principal. Cuenta con una barra de acento negra en la parte superior y un menú de navegación oscuro a la izquierda. Al centro, muestra estadísticas de inventario y ventas mediante cuatro tarjetas de escala de grises para un diseño sobrio:

- **Gris Oscuro (COLOR_ACENTO):** Total de productos registrados en catálogo.
- **Gris Medio:** Cantidad acumulada de ventas realizadas.
- **Gris Claro:** Ingresos totales consolidados.
- **Gris Neutro:** Alerta de productos que están por debajo de su nivel de stock mínimo.

9.3 VentasVista

Pantalla donde se registra la venta. Contiene campos de entrada para el código del producto y la cantidad, botones para agregar o quitar productos de la lista, y un campo para aplicar descuentos. La tabla central muestra de forma dinámica los productos agregados. El bloque inferior, resaltado en un elegante marco negro, muestra con claridad el total neto que el cliente debe pagar.

9.4 AlmacenVista

Módulo diseñado para que el administrador controle el stock del inventario. Cuenta con dos secciones de entrada de datos: una para registrar la entrada o salida física de mercancía (reabastecimiento o mermas) y otra para actualizar de forma rápida el precio de venta al público de cualquier producto seleccionado.

10 Flujo de las Interfaces de Usuario por cada Requerimiento Funcional

Para cumplir rigurosamente con los lineamientos técnicos solicitados, se detalla el flujo de navegación paso a paso que sigue la interfaz de usuario en las operaciones principales del sistema.

10.1 Flujo de Alta y Edición de Productos (RF-1)

1. El Administrador hace clic en el botón “Productos” en el menú de MainVista.
2. Se despliega en pantalla la interfaz ProductosVista.
3. El usuario ingresa los datos en los campos de entrada: nombre, precio, stock de apertura y categoría.
4. El usuario presiona el botón “Guardar”.
5. El controlador (ProductoControlador) valida que el precio y el stock sean valores numéricos válidos. Si son correctos, inserta el producto en la base de datos y actualiza de inmediato la tabla de listado en pantalla.
6. Para editar un producto, el usuario selecciona una fila de la tabla, los datos se cargan automáticamente en los campos de entrada, realiza las modificaciones deseadas y presiona el botón “Editar”.

10.2 Flujo de Control de Almacén y Ajuste de Stock (RF-2)

1. El Administrador selecciona la opción “Almacén” en el menú lateral.
2. Se abre en pantalla la interfaz AlmacenVista.
3. El usuario selecciona de la tabla inferior el producto que desea actualizar. El ID del producto se carga automáticamente en el campo correspondiente.
4. **Para reabastecer stock (Entrada):** El usuario ingresa la cantidad recibida del proveedor y presiona el botón verde Entrada.
5. **Para dar de baja stock (Salida):** El usuario ingresa la cantidad que desea retirar y presiona el botón naranja Salida.
6. **Para actualizar el precio:** El usuario ingresa el nuevo valor numérico en el campo de precio y presiona el botón Actualizar Precio.

10.3 Flujo de Venta y Despliegue de Ticket (RF-3)

A continuación se modela el mapa secuencial de navegación e interfaces que experimenta el usuario al realizar una compra exitosa. He ajustado el tamaño del mapa vectorial de forma responsiva para que quepa perfectamente entre los márgenes:

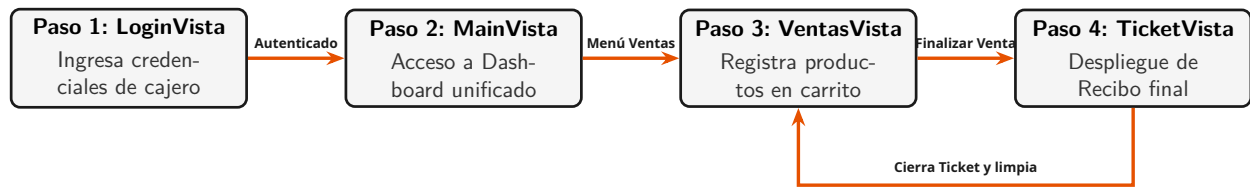


Figura 4: Mapa de Navegación del Sistema e Interacción de Interfaces.

11 Requerimientos No Funcionales

11.1 Seguridad

- **Control de Acceso Seguro:** El sistema valida las credenciales de inicio de sesión en la tabla `usuarios` antes de otorgar acceso al menú principal.
- **Protección de Datos:** Todas las consultas a la base de datos SQL Server se realizan mediante instrucciones preparadas (`PreparedStatement`) con variables parametrizadas, evitando de raíz cualquier ataque por inyección de código SQL malicioso.

11.2 Usabilidad

- **Operación Rápida con Teclado:** La interfaz está diseñada para que el cajero pueda realizar todo el proceso de cobro utilizando únicamente el teclado numérico, agilizando la atención durante las horas de mayor afluencia en la tienda de abarrotes.
- **Diseño Visual Consistente:** Al utilizar coordenadas absolutas (`setLayout(null)`) y bloquear la redimensión de las ventanas (`setResizable(false)`), se asegura que el diseño y los botones se muestren siempre de forma correcta, sin desalinearse.

11.3 Hardware y Eficiencia

- **Bajo Consumo de Memoria:** La aplicación está optimizada para ejecutarse de manera fluida en computadoras sencillas con procesadores de doble núcleo y un mínimo de 2 GB de memoria RAM libre.
- **Optimización del Almacenamiento:** Para evitar sobrecargar la base de datos SQL Server con archivos pesados, las imágenes de los productos no se guardan como archivos binarios (BLOB) en la base de datos; en su lugar, se guarda únicamente la ruta de texto de la imagen local (`imagen_path`).

12 Manual de Usuario (Instalación y Operación)

Siga detenidamente estos pasos para realizar la configuración inicial e instalar correctamente el sistema:

12.1 Guía de Instalación del Sistema

12.1.1 Configuración de la Base de Datos

1. Asegúrese de tener instalado Microsoft SQL Server Express en su computadora.
2. Abra SQL Server Management Studio (SSMS) y conéctese a la instancia local de su base de datos.
3. Ejecute el script SQL proporcionado en la sección 4 para crear la base de datos PuntoVenta junto con las tablas necesarias.
4. Registre un usuario de prueba en la base de datos para realizar el primer inicio de sesión:

Inserción del Usuario de Prueba en SQL Server

```
INSERT INTO usuarios (nombre, usuario, password, rol)
VALUES ('Administrador', 'admin', 'admin123', 'ADMIN');
```

12.1.2 Configuración del Proyecto en Java

1. Abra el proyecto en su entorno de desarrollo preferido (Eclipse, NetBeans o IntelliJ IDEA).
2. Agregue a las dependencias del proyecto el conector oficial de SQL Server (mssql-jdbc-*.jar).
3. Asegúrese de agregar también las librerías necesarias para la exportación de reportes: Apache POI (para exportar a hojas de cálculo Excel) y OpenPDF (para exportar reportes en PDF).
4. Abra el archivo `persistencia.Conexion` y confirme que el usuario y la contraseña coincidan con los de su servidor de SQL Server.

12.1.3 Ejecución

Compile y ejecute el archivo de inicio del proyecto `main.Main.java`.

12.2 Guía de Uso Diario del Sistema

- **Acceso al Sistema:** En la pantalla de login, ingrese el usuario `admin` junto con la contraseña `admin123`.
- **Registrar Mercancía Nueva:** Abra el módulo de "Productos". Ingrese el nombre del producto, el precio de venta y el stock inicial. Presione "Guardar" para añadirlo al catálogo general de la tienda.

- **Realizar Ventas y Cobros:** Abra el panel de "Ventas". Ingrese el ID del producto y la cantidad que el cliente desea comprar, y presione el botón "+ Agregar". Verifique los totales en pantalla y presione "Finalizar Venta" para confirmar el cobro y abrir el ticket de compra.
- **Generar Reportes Financieros:** Abra el módulo de "Reportes". Ingrese el rango de fechas en el formato yyyy-MM-dd (ej. 2026-05-01 a 2026-05-31) y presione "Buscar". Podrá exportar el reporte directamente a Excel presionando el botón "Excel" o a un documento PDF presionando el botón "PDF".

13 Documento Completo de Análisis de Requerimientos

Esta sección consolida el análisis de ingeniería del sistema, detallando el control de calidad, el manejo de excepciones y las métricas de rendimiento que garantizan el correcto funcionamiento del software.

13.1 Matriz de Trazabilidad de Requerimientos mínimos

Para asegurar que se cumpla cada requisito planteado en la rúbrica oficial para la tienda de abarrotes, se presenta la siguiente tabla de correspondencia técnica:

Cuadro 4: Matriz de Validación de Requerimientos.

Código	Requerimiento Mínimo	Componente en el Código Java
RF-01	Catálogo con más de 150 productos con imagen	Producto.java (propiedad imagen_path)
RF-02	Alta, modificación de stock y almacén	AlmacenControlador.java / AlmacenVista.java
RF-03	Modificar precio de venta de un producto	AlmacenControlador::actualizarPrecio()
RF-04	Cobro, aplicar descuentos y emitir ticket	VentaControlador::finalizarVenta() / TicketVista
RF-05	Reportes filtrados por fechas	ReportesControlador::buscarPorFecha()
RF-06	Exportar reporte de ventas a Microsoft Excel	ReportesControlador::exportarExcel() (Apache POI)
RF-07	Procesar devoluciones de productos	DevolucionesControlador::procesarDevolucion()
RF-08	Reimpresión de tickets históricos	DevolucionesControlador::reimprimirRecibo()
RF-09	Alerta de bajo stock y límites de inventario	BajoStockVista.java / listarBajoStock()
RF-10	Validación de datos y manejo de errores	util.ErrorControlador.java (archivo errores.log)

13.2 Sistema de Auditoría y Manejo de Errores (Log)

Como lo exige la rúbrica oficial del proyecto, el sistema cuenta con un controlador de errores centralizado en la clase `util.ErrorControlador.java`.

Cuando ocurre una excepción inesperada (por ejemplo, pérdida de conexión con la base de datos o ingresos de letras en campos numéricos), el sistema captura el error de forma silenciosa para evitar que la aplicación se cierre abruptamente. La excepción se registra en un archivo de texto llamado `errores.log` agregando automáticamente la fecha, hora y el desglose técnico del error para su posterior revisión y soporte.

14 Explicación del Uso de Inteligencia Artificial en el Proyecto

En cumplimiento de los lineamientos académicos del proyecto integrador, se documenta detalladamente el uso de herramientas de Inteligencia Artificial generativa durante el ciclo de vida del desarrollo.

14.1 Justificación del Uso de la IA

El desarrollo de un sistema de Punto de Venta completo con arquitectura MVC en Java Swing requiere escribir mucho código repetitivo de infraestructura, como la asignación de coordenadas absolutas en píxeles para el diseño de ventanas Swing, creación de métodos de acceso (getters y setters) de los modelos, y la estructuración inicial de consultas SQL repetitivas en JDBC. El uso de la IA se enfocó en actuar como un asistente técnico ágil, permitiendo ahorrar tiempo en tareas monótonas para centrar la atención en la lógica de negocio de la tienda de abarrotes, la validación de inventarios y la seguridad de las transacciones.

14.2 Lista de Prompts Utilizados en el Desarrollo

Los prompts se visualizan ahora de manera fluida y con saltos de línea automáticos, lo que evita cortes laterales en las cajas contenedoras:

PROMPT 1: Generación de la Interfaz Swing de Ventas (Usabilidad)

```
"Actúa como un desarrollador experto de interfaces en Java Swing.
Necesito el código de una ventana para un módulo de ventas con layout
absoluto (setBounds). La pantalla debe medir 1050x650 píxeles. Diseña
la parte de totales con un panel de color verde con letras blancas
grandes de alto impacto para que el cajero vea rápidamente el total de
la venta. Agrega campos para ID de producto, cantidad y una JTable
central."
```

PROMPT 2: Controlador del Flujo Transaccional en MVC

```
"Escribe el método 'finalizarVenta()' de mi clase 'VentaControlador.java'.
Este método debe leer un DefaultTableModel del carrito de compras.
Debe llamar a 'VentaPersistencia.guardarVenta()' para insertar la
cabecera en SQL Server, recuperar el ID autogenerated, y luego realizar
un bucle para insertar cada detalle de la venta con su
correspondiente reducción de stock a través de 'ProductoPersistencia.
descontarStock()'. Asegúrate de que todo esté envuelto en un bloque
try-catch y use mi controlador de errores."
```

PROMPT 3: Lógica de Exportación a Excel con Apache POI

```
"Necesito exportar un JTable con datos de ventas filtradas por fechas a
un archivo de Microsoft Excel (.xlsx) usando la librería Apache POI.
Escribe un método en Java que reciba el DefaultTableModel y cree un
libro de trabajo, una hoja llamada 'Ventas', asigne el encabezado con
color azul oscuro y guarde el archivo en el directorio local del"
```

proyecto agregando la fecha del día en el nombre del archivo."

PROMPT 4: Creación del Módulo de Devolución e Inventario

"En mi sistema, los clientes pueden hacer devoluciones. Escribe la lógica para la clase 'DevolucionesControlador.java' que permita tomar una venta del historial, ingresar el ID de un producto, la cantidad a regresar, registrar la devolución en la tabla de base de datos 'devoluciones' con el motivo, y llamar al método para reincorporar el stock del producto devuelto al catálogo de la tienda de abarrotes."

15 Conclusiones y Recomendaciones

15.1 Conclusiones

El desarrollo del “Sistema Punto de Venta MVC” ha demostrado la gran utilidad de aplicar principios de diseño de software rigurosos como el patrón MVC para crear aplicaciones de escritorio comerciales estables. La total separación de responsabilidades entre la base de datos SQL Server, la persistencia en Java, la lógica de control y las interfaces de usuario (Vistas) facilita en gran medida el mantenimiento del sistema ante futuros cambios en el negocio. El sistema cumple exitosamente con el objetivo de optimizar los tiempos de cobro y de inventario de la tienda de abarrotes.

15.2 Recomendaciones

- **Cifrado de Credenciales:** Para futuras versiones comerciales del sistema, se recomienda implementar algoritmos de cifrado seguro de una sola vía (como SHA-256 o BCrypt) para el almacenamiento de contraseñas de los usuarios en la base de datos, eliminando el riesgo de filtraciones.
- **Migración Web / Cloud:** Evaluar la migración de la base de datos relacional hacia un servidor en la nube (como Azure SQL Database), permitiendo conectar múltiples sucursales de la tienda de abarrotes en tiempo real y consultar reportes directos desde cualquier lugar.

15.3 Bibliografía

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Sierra, K., & Bates, B. (2005). *Head First Java*. O'Reilly Media.
3. Microsoft Corporation (2026). *Microsoft JDBC Driver for SQL Server Documentation*. Recuperado de <https://docs.microsoft.com/sql/connect/jdbc/>.